Project 1: Smart Recruitment & Applicant Tracking System (ATS)

Web Development Internship — Project Requirements and Four-Week Implementation Plan

Industry: HR Tech / SaaS. Build a recruitment platform where companies manage job openings, applicants, interviews, offer letters, and onboarding. The system should provide role-specific workflows for administrators, HR staff, and candidates, with clear application status tracking and hiring analytics.

System Context and Problem Statement

Recruitment teams often manage job posts, resumes, interview schedules, and offer decisions across disconnected tools. This can create duplicate records, missed follow-ups, inconsistent candidate communication, and limited visibility into hiring performance. Build a secure web application that centralizes the hiring lifecycle from job publication through offer and onboarding, while enforcing role-based access and maintaining a traceable history of application-stage changes.

System Objectives and Success Metrics

Objective Area	Target Objective
Role-specific access	Provide separate Admin, HR, and Candidate dashboards with permissions enforced on the server, not only hidden in the interface.
Job and application workflow	Allow authorized staff to create, edit, publish, close, and archive jobs; allow candidates to submit applications through validated online forms.
Pipeline visibility	Track candidates through Applied → Screening → Interview → Selected → Offered, with controlled transitions and an audit trail.
Communication and scheduling	Support interview scheduling and email notifications for meaningful application and interview events, with delivery status and retry handling where feasible.
Offer and onboarding support	Generate offer letters from approved templates, record offer status, and track next steps without exposing other candidates' data.
Hiring analytics	Summarize job openings, applications, stage conversion, time-to-hire where dates exist, and offer acceptance where captured.
Security and reliability	Protect resume files and personal information, validate inputs, and test authentication, authorization, and file-upload controls.

Technology Stack

Frontend: React or a comparable component-based framework, HTML5, CSS, and JavaScript/TypeScript. Backend: Node.js with Express or an equivalent REST API framework. Database: PostgreSQL or MySQL. Authentication: secure sessions or token-based authentication with appropriate expiration and password hashing. File storage: private local storage for development or a controlled object-storage service. Email: SMTP or a transactional email provider. Testing: unit/integration tests and browser-based end-to-end tests. Use environment variables for secrets and document the selected stack in README.md.

Data Requirements and Assumptions

Entities should include users, roles, companies or organizations, jobs, candidate profiles, applications, resume metadata, interview events, interview feedback, offer records, email/notification events, and onboarding tasks. Define primary/foreign keys, status enums, timestamps, and ownership relationships. Resume files should be stored privately; store metadata and a controlled file reference in the database. Use synthetic candidate data for demonstrations. Do not use real resumes or personal data in a public repository. Document retention/deletion assumptions and avoid collecting fields that are not needed.

Core KPI Definitions and Evaluation Metrics

Metric	Definition / Evaluation Guidance
Application completion rate	Completed applications divided by application starts, if starts are tracked; otherwise report completed application count only.
Pipeline conversion	Candidates entering a stage divided by candidates entering the preceding stage, using a documented cohort and time window.
Time-to-hire	Days between application or requisition approval and offer acceptance/hire, only when the required dates are captured. State the chosen definition.
Interview scheduling reliability	Scheduled interviews with valid date/time and assigned participants; track cancellations or reschedules if implemented.
Notification success	Successful email deliveries divided by attempted deliveries, based on provider responses or a clearly labeled simulated mode.
Access-control coverage	Automated tests confirming each role can perform allowed actions and is denied unauthorized actions.
Application data integrity	Valid application records with required references, allowed statuses, and consistent timestamps.

Four-Week Development Lifecycle

Week 1: Requirements, Architecture & Core Data Model

Engineering Focus: Establish requirements, architecture, data structures, and a reproducible development environment.

Technical Implementation Details: Define user journeys for Admin, HR, and Candidate roles. Create wireframes, route map, API contract, database schema, and permission matrix. Set up the repository, environment configuration, linting, and test framework. Implement authentication foundations and seed synthetic users, jobs, and candidate records.

Deliverables: Requirements document, wireframes, ER diagram, permission matrix, initial database migrations, project skeleton, and setup instructions.

Week 2: Job Posting, Candidate Applications & Pipeline

Engineering Focus: Implement the primary user-facing workflows with secure server-side validation.

Technical Implementation Details: Build job listing/detail pages, job management forms, candidate profile, resume upload validation, and application submission. Implement server-side validation, secure file handling, and pipeline stage transitions. Add search/filter for candidates and applications, with pagination and empty/error states.

Deliverables: Working job and application workflows, validated upload flow, candidate search, pipeline UI, API tests, and documented status-transition rules.

Week 3: Interviews, Notifications & Offer Letters

Engineering Focus: Add collaboration or business-process features and integrate supporting services.

Technical Implementation Details: Implement interview creation, scheduling, rescheduling/cancellation, and candidate/HR views. Trigger email notifications for application receipt, interview updates, and offer status where configured. Generate offer letters from a controlled template and record draft/issued/accepted/declined states. Add activity history and authorization tests.

Deliverables: Interview calendar/workflow, notification service or clearly documented mock, offer generation, activity log, and integration tests.

Week 4: Analytics, Security Hardening & Deployment

Engineering Focus: Complete analytics, testing, security review, deployment, and documentation.

Technical Implementation Details: Build hiring analytics using accurately defined counts and rates. Add filters by job, date, and status; handle empty datasets and inconsistent dates. Review authentication,

authorization, upload limits, file access, validation, secrets, and error handling. Test key end-to-end journeys, deploy a demo, and prepare documentation and presentation.

Deliverables: Admin/HR analytics dashboard, test report, deployed demo or deployment guide, README, sample data, and final presentation.

Dashboard / Application Scope

Suggested pages: (1) Admin Overview—users, roles, company configuration, and system activity; (2) Job Management—create/edit/publish/close jobs; (3) Candidate Dashboard—profile, resume, applications, and current status; (4) HR Pipeline—candidate search, filters, stage transitions, and activity history; (5) Interviews—calendar/list, participant details, and rescheduling; (6) Offers & Onboarding—template-based offer documents and onboarding checklist; (7) Hiring Analytics—application volume, stage conversion, time-to-hire where measurable, and offer outcomes.

Repository Structure and Reproducibility

Use a structure such as frontend/, backend/ or src/, database/migrations/, database/seed/, tests/, docs/, and uploads/ (ignored by Git). Keep uploaded resumes and secrets out of version control. Provide .env.example without real credentials. Include API documentation, schema/migrations, synthetic seed data, test commands, and setup/deployment instructions.

Final Deliverable

A working recruitment and applicant tracking web application with separate Admin, HR, and Candidate experiences; job posting and application workflows; a traceable candidate pipeline; resume upload/search; interview scheduling; email notification support; offer-letter generation; application status tracking; and hiring analytics. Include source code, database setup, tests, synthetic demo data, documentation, and a short technical presentation.

Universal Analytical Standards

Validate all user input on the server and apply appropriate client-side validation for usability. Enforce authorization at the API/data layer, not only in the interface. Use consistent status definitions, timestamps, and timezone rules; handle empty states and errors clearly. Protect personal information, uploaded files, credentials, and secrets. Use synthetic demonstration data, document assumptions and limitations, and avoid claiming that mocked integrations are production-connected.

Version Control and Work Discipline

Use Git and GitHub to maintain a meaningful, incremental record of development across all four weeks. Commit logical changes with clear messages for architecture, data model, core workflows, integrations, testing, and UI improvements. Keep secrets, local environment files, uploaded documents, payment credentials, and production data out of version control. Review changes before merging and keep the main branch runnable.

Final Submission Requirements

Submit a GitHub repository containing source code, database migrations/seeds, automated tests, and setup instructions; a README describing architecture, features, roles, environment configuration, and limitations; screenshots or a deployed demo; test evidence; and a final report explaining design decisions, security controls, known gaps, and future improvements. Include only synthetic demo data and provide an .env.example without real secrets.

Technical Presentation and Evaluation

Evaluate role separation and server-side authorization, completeness of the recruitment lifecycle, usability of pipeline/search/scheduling flows, secure handling of resumes and personal data, data integrity, notification and offer workflows, test coverage, deployment/reproducibility, and the ability to explain limitations. Analytics must use defined denominators and date rules; do not claim real email delivery if the demo uses a mock service.

Project 2: Project Management & Team Collaboration SaaS

Web Development Internship — Project Requirements and Four-Week Implementation Plan

Industry: IT Services / Enterprise Software. Build a web application where teams plan projects, assign tasks, track deadlines, and monitor work progress. The platform should support organizations, workspaces, teams, projects, task collaboration, notifications, and productivity reporting with role-based access.

System Context and Problem Statement

Teams need a shared view of ownership, priorities, deadlines, dependencies, and work status. When project information is scattered across messages and spreadsheets, overdue tasks and blockers are difficult to identify, and managers lack reliable progress reporting. Build a multi-organization

collaboration platform that keeps project and task activity in one place, supports practical workflows, and ensures users can only access resources permitted by their organization and role.

System Objectives and Success Metrics

Objective Area	Target Objective
Organization and workspace structure	Support organizations, workspaces, teams, and projects with clear ownership and membership rules.
Task lifecycle	Create, assign, prioritize, schedule, and update tasks and subtasks; support a Kanban workflow with drag-and-drop interactions and accessible alternative controls.
Collaboration history	Support comments, file attachments, and activity history so teams can understand changes and decisions.
Notifications and deadlines	Notify relevant users about assignments, comments, due-date changes, and overdue tasks, avoiding duplicate or noisy alerts where possible.
Progress visibility	Show project progress, task status distribution, overdue work, workload indicators, and trends with documented calculation rules.
Role-based access	Enforce permissions for owners, managers, and employees at API and data-access layers.
Maintainability and quality	Provide responsive layouts, reliable task updates, tests, and clear deployment/reproducibility instructions.

Technology Stack

Frontend: React or equivalent, TypeScript recommended, and a responsive CSS framework or design system. Backend: Node.js/Express or equivalent REST API. Database: PostgreSQL or MySQL. Optional real-time updates: WebSocket or server-sent events; a polling-based implementation is acceptable if documented. File attachments: controlled storage with type/size validation. Notifications: in-app notification records plus optional email. Testing: unit, API integration, and browser end-to-end tests. Use environment variables for database credentials and provider secrets.

Data Requirements and Assumptions

Core entities may include organizations, memberships, users, workspaces, teams, projects, project members, tasks, subtasks, labels, comments, attachments, activity events, notifications, and status/priority definitions. Each organization-owned record must include a trustworthy organization or parent-resource relationship. Model task assignees, due dates, completion timestamps, and parent-child relationships. Define how progress is calculated (for example, completed tasks divided by all non-archived tasks) and how archived/cancelled tasks are treated. Use synthetic workspaces and sample files in the demo.

Core KPI Definitions and Evaluation Metrics

Metric	Definition / Evaluation Guidance
Task completion rate	Completed eligible tasks divided by total eligible tasks in a stated project and time window; document treatment of archived/cancelled tasks.
On-time completion rate	Tasks completed on or before due date divided by tasks completed with valid due dates.
Overdue task count	Open tasks with a due date earlier than the current date/time, using the application's stated timezone rule.
Project progress	A defined calculation such as completed non-archived tasks divided by all non-archived tasks; optionally support weighted progress if documented.
Assignment coverage	Tasks with an assigned owner divided by all active tasks.
Notification processing	Notifications created and marked delivered/read where implemented; distinguish in-app state from actual email delivery.
Authorization integrity	Tests demonstrate that users cannot read or mutate projects, tasks, comments, or attachments outside their organization or permissions.

Four-Week Development Lifecycle

Week 1: Product Scope, UX & Multi-Tenant Foundation

Engineering Focus: Establish requirements, architecture, data structures, and a reproducible development environment.

Technical Implementation Details: Map organization, workspace, team, project, and task relationships. Design navigation and role permissions for owner, manager, and employee. Create database migrations, authentication, membership management, and organization-scoped API middleware. Build responsive application shell and synthetic seed data.

Deliverables: Wireframes, data model, permission matrix, authenticated application shell, organization/workspace foundation, and initial automated tests.

Week 2: Projects, Tasks & Kanban Workflow

Engineering Focus: Implement the primary user-facing workflows with secure server-side validation.

Technical Implementation Details: Implement project creation and membership, task CRUD, assignment, priorities, due dates, subtasks, labels, and status transitions. Add a Kanban board with drag-and-drop and keyboard/accessibility fallback. Validate that all task operations are scoped to authorized projects and organizations.

Deliverables: Working project/task workflows, Kanban board, task filters, server-side validation, and API/UI tests.

Week 3: Collaboration, Attachments & Notifications

Engineering Focus: Add collaboration or business-process features and integrate supporting services.

Technical Implementation Details: Add comments, controlled file attachments, activity events, in-app notifications, and optional email notifications. Implement overdue-task detection based on documented date/time rules. Prevent unauthorized attachment downloads and validate upload type, size, and storage location.

Deliverables: Comment and activity history, secure attachment workflow, notification center, overdue alerts, and integration tests.

Week 4: Analytics, Quality Assurance & Deployment

Engineering Focus: Complete analytics, testing, security review, deployment, and documentation.

Technical Implementation Details: Build project dashboards and reports for completion, overdue tasks, workload, and activity. Test role permissions, concurrent updates, empty states, responsive layouts, and task movement. Add error logging and deployment configuration; document limitations such as simulated notifications or non-real-time refresh.

Deliverables: Project analytics dashboard, end-to-end test results, deployed demo or deployment guide, README, sample organization, and final presentation.

Dashboard / Application Scope

Suggested pages: (1) My Work—assigned tasks, upcoming deadlines, and overdue items; (2) Organization/Workspace—membership and team management; (3) Project Overview—progress, milestones, and project members; (4) Kanban Board—status columns, task cards, priorities, and due dates; (5) Task Details—subtasks, comments, attachments, and activity history; (6) Notifications—assignment, comment, due-date, and overdue alerts; (7) Reports—completion rates, overdue work, workload, and progress trends.

Repository Structure and Reproducibility

Use a structure such as frontend/, backend/ or src/, database/migrations/, database/seed/, tests/, docs/, and storage/ (ignored by Git if local). Keep organization scoping in reusable backend authorization functions. Include .env.example, API documentation, seed scripts, migration instructions, test commands, and deployment notes. Do not commit uploaded files, credentials, or production data.

Final Deliverable

A usable team collaboration SaaS prototype supporting organizations, workspaces, teams, projects, Kanban task management, assignment/priorities/deadlines/subtasks, comments, attachments, activity

history, notifications, overdue alerts, role-based access, and productivity analytics. Submit code, schema, tests, sample data, documentation, a running demo or deployment instructions, and presentation.

Universal Analytical Standards

Validate all user input on the server and apply appropriate client-side validation for usability. Enforce authorization at the API/data layer, not only in the interface. Use consistent status definitions, timestamps, and timezone rules; handle empty states and errors clearly. Protect personal information, uploaded files, credentials, and secrets. Use synthetic demonstration data, document assumptions and limitations, and avoid claiming that mocked integrations are production-connected.

Version Control and Work Discipline

Use Git and GitHub to maintain a meaningful, incremental record of development across all four weeks. Commit logical changes with clear messages for architecture, data model, core workflows, integrations, testing, and UI improvements. Keep secrets, local environment files, uploaded documents, payment credentials, and production data out of version control. Review changes before merging and keep the main branch runnable.

Final Submission Requirements

Submit a GitHub repository containing source code, database migrations/seeds, automated tests, and setup instructions; a README describing architecture, features, roles, environment configuration, and limitations; screenshots or a deployed demo; test evidence; and a final report explaining design decisions, security controls, known gaps, and future improvements. Include only synthetic demo data and provide an .env.example without real secrets.

Technical Presentation and Evaluation

Evaluate the quality of multi-tenant data isolation, role enforcement, task lifecycle, Kanban usability, collaboration history, secure attachments, deadline/notification logic, analytical definitions, responsive behavior, testing, and deployment. Demonstrate at least two organizations and prove that one organization’s users cannot access another organization’s data.

Project 3: E-commerce & Inventory Management Platform

Industry: Retail Tech / E-commerce. Build a complete online store connected to an inventory and order management dashboard. Customers should be able to discover products, manage a cart, check out, pay through a gateway or sandbox, and track orders, while administrators manage stock, orders, coupons, reviews, and sales analytics.

System Context and Problem Statement

Online stores must keep product listings, available inventory, checkout, payments, and order status synchronized. Weak inventory controls can lead to overselling, while incorrect payment handling or incomplete order workflows can create duplicate orders, unclear refunds, and unreliable revenue reporting. Build a customer storefront and a protected administrative interface with consistent order state transitions, stock adjustments, and verifiable transaction records. Use a payment sandbox for demonstrations unless a real merchant integration is explicitly approved.

System Objectives and Success Metrics

Objective Area	Target Objective
Product discovery	Provide product catalog, categories, search, filters, product details, and responsive browsing.
Customer and cart workflow	Support accounts, persistent cart behavior, checkout validation, and order confirmation.
Payment integrity	Integrate a gateway sandbox or documented mock; verify payment status server-side and never trust a client-side success redirect alone.
Inventory management	Track stock changes, reserve or decrement inventory consistently, and alert administrators when stock crosses a configured threshold.
Order lifecycle	Support order processing, cancellation, shipment/tracking updates, and refund workflow states with clear permissions and audit history.
Store engagement	Implement coupons, product reviews, and order tracking with validation and abuse prevention appropriate to the prototype.
Business reporting	Provide sales, order, revenue, product, and inventory analytics using explicit definitions and reliable transaction states.

Technology Stack

Frontend: React or equivalent, HTML/CSS, and JavaScript/TypeScript. Backend: Node.js with Express or equivalent. Database: PostgreSQL or MySQL. Authentication: secure sessions or token-based authentication and password hashing. Payments: a provider sandbox such as Stripe/Razorpay test mode, or a clearly labeled mock if provider access is unavailable. Product images and other uploads should use

controlled storage. Testing: unit, API integration, payment-webhook simulation, and browser end-to-end tests. Keep API keys in environment variables and never store raw card details.

Data Requirements and Assumptions

Core entities may include users, addresses, products, categories, product images, inventory movements, carts, cart items, orders, order items, payment attempts/transactions, refunds, coupons, coupon redemptions, reviews, and shipment/tracking events. Store price snapshots and product descriptions on order items so historical orders remain interpretable after catalog changes. Use integer minor currency units or a suitable fixed-precision decimal representation; do not use floating-point arithmetic for monetary totals. Define inventory reservation/adjustment rules, order statuses, payment statuses, cancellation eligibility, and refund state transitions. Use synthetic customers and sandbox transactions.

Core KPI Definitions and Evaluation Metrics

Metric	Definition / Evaluation Guidance
Order conversion	Completed orders divided by eligible checkout starts or sessions, only if the denominator is tracked; otherwise report order counts without implying a conversion rate.
Gross sales / net revenue	Define treatment of discounts, taxes, shipping, cancellations, and refunds; exclude failed or unpaid orders from paid-sales totals.
Average order value	Eligible paid order value divided by number of eligible paid orders for the selected period.
Inventory availability	Products/SKUs currently above zero available stock divided by active products/SKUs; state whether variants are counted separately.
Low-stock exposure	Count of active SKUs at or below their configured reorder threshold.
Payment verification integrity	Test that an order becomes paid only after verified server-side provider confirmation or a clearly documented sandbox/mock equivalent.
Order fulfilment / cancellation	Counts and rates of orders by defined status; distinguish cancelled, refunded, paid, and shipped orders.

Four-Week Development Lifecycle

Week 1: Store Architecture, Catalog & Data Model

Engineering Focus: Establish requirements, architecture, data structures, and a reproducible development environment.

Technical Implementation Details: Define customer and admin journeys, product/category schema, cart and order entities, inventory movement rules, and status diagrams. Set up frontend/backend, migrations, seed data, authentication, and protected admin routes. Build responsive catalog, product detail, search, and category/filter experiences.

Deliverables: Wireframes, database schema, seed catalog, project skeleton, customer/admin route protection, and working product browsing.

Week 2: Cart, Checkout & Payment Workflow

Engineering Focus: Implement the primary user-facing workflows with secure server-side validation.

Technical Implementation Details: Implement cart operations, quantity validation, price calculation on the server, address/checkout forms, and order creation. Integrate a payment sandbox or clearly labeled mock. Verify gateway callbacks/webhooks where available, make payment handling idempotent, and prevent client-controlled prices or success flags from marking orders paid.

Deliverables: Working cart and checkout, order records, sandbox/mock payment flow, payment state handling, and tests for invalid/duplicate payment events.

Week 3: Inventory, Orders & Store Features

Engineering Focus: Add collaboration or business-process features and integrate supporting services.

Technical Implementation Details: Implement stock adjustments or reservations, low-stock thresholds, order processing, cancellation eligibility, refund states, coupons, reviews, and order tracking. Use database transactions or appropriate consistency controls for stock and order changes. Record inventory movements and order activity, and protect admin-only operations.

Deliverables: Admin inventory/order dashboard, low-stock alerts, coupon/review features, order tracking, and inventory/order workflow tests.

Week 4: Analytics, Security & Deployment

Engineering Focus: Complete analytics, testing, security review, deployment, and documentation.

Technical Implementation Details: Build sales/revenue/order analytics with documented filters and exclusions. Test pricing, coupon validation, stock races, payment verification, authorization, review validation, and cancellation/refund transitions. Configure deployment, environment variables, error handling, and demo seed data. Prepare a walkthrough with sandbox transactions and known limitations.

Deliverables: Storefront and admin analytics dashboard, security/test report, deployed demo or deployment guide, README, sandbox test evidence, and final presentation.

Dashboard / Application Scope

Suggested pages: (1) Storefront—catalog, categories, search, filters, and product details; (2) Cart & Checkout—quantities, validated totals, address, and payment status; (3) Customer Account—orders, tracking, and reviews; (4) Inventory Admin—SKU stock, inventory movements, reorder thresholds, and low-stock alerts; (5) Order Management—status updates, cancellations, and refund workflow; (6) Promotions & Reviews—coupon setup/redemption and review moderation; (7) Sales Analytics—paid

orders, revenue definitions, average order value, product performance, cancellations/refunds, and inventory exposure.

Repository Structure and Reproducibility

Use a structure such as frontend/, backend/ or src/, database/migrations/, database/seed/, tests/, docs/, and private uploads/storage excluded from Git. Keep payment keys and webhook secrets in environment variables. Include .env.example, payment sandbox setup, webhook testing steps, seed data, schema/migration instructions, test commands, and deployment notes. Never store or log raw card details or production customer data.

Final Deliverable

A full-stack e-commerce prototype with product catalog/search/filtering, customer accounts, cart and checkout, sandbox/mock payment verification, inventory tracking and low-stock alerts, order processing/cancellation/refund states, coupons, reviews, order tracking, admin analytics, transaction records, tests, documentation, and deployment instructions.

Universal Analytical Standards

Validate all user input on the server and apply appropriate client-side validation for usability. Enforce authorization at the API/data layer, not only in the interface. Use consistent status definitions, timestamps, and timezone rules; handle empty states and errors clearly. Protect personal information, uploaded files, credentials, and secrets. Use synthetic demonstration data, document assumptions and limitations, and avoid claiming that mocked integrations are production-connected.

Version Control and Work Discipline

Use Git and GitHub to maintain a meaningful, incremental record of development across all four weeks. Commit logical changes with clear messages for architecture, data model, core workflows, integrations, testing, and UI improvements. Keep secrets, local environment files, uploaded documents, payment credentials, and production data out of version control. Review changes before merging and keep the main branch runnable.

Final Submission Requirements

Submit a GitHub repository containing source code, database migrations/seeds, automated tests, and setup instructions; a README describing architecture, features, roles, environment configuration, and limitations; screenshots or a deployed demo; test evidence; and a final report explaining design decisions, security controls, known gaps, and future improvements. Include only synthetic demo data and provide an .env.example without real secrets.

Technical Presentation and Evaluation

Evaluate catalog and checkout usability, server-side price calculations, payment verification, inventory consistency, order/refund state handling, security of customer/admin routes, correct financial metric definitions, test coverage, and reproducibility. Demonstrate successful, failed, and duplicate payment-event scenarios in sandbox/mock mode and explain what remains simulated.

